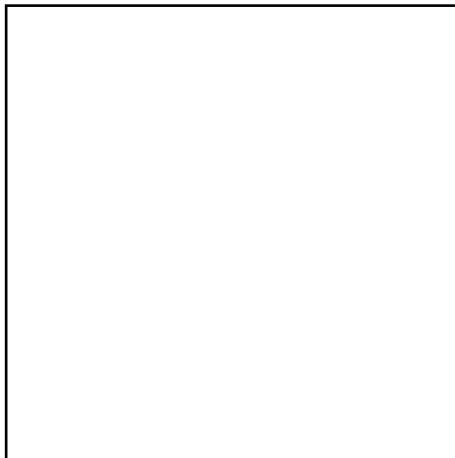


LAPORAN PERTEMUAN
PRAKTIKUM STRUKTUR DATA DAN ALGORITMA
PERTEMUAN 4
IMPLEMENTASI STACK DAN QUEUE BERBASIS LINKED LIST



Disusun Oleh:

Nama : Atalariq Barra Hadinugraha
NIM : 25/557554/SV/26192
Kelas : PSDA - B2
Dosen Pengampu : Dr. John Doe

D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

A. Tujuan Praktikum	3
B. Dasar Teori	3
1. Abstract Data Type ADT	3
2. Stack	3
3. Queue	3
4. Linked List sebagai Basis Implementasi	3
C. Hasil dan Pembahasan	4
1. Implementasi Node	4
2. Implementasi Stack	4
3. Implementasi Queue	4
4. Analisis Kompleksitas	5
D. Kesimpulan	5
Lampiran	6
Daftar Pustaka	7

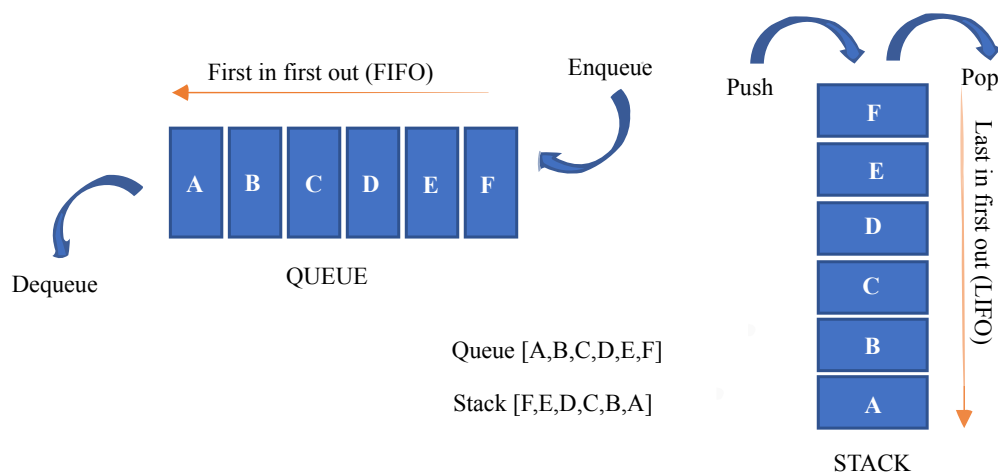
A. Tujuan Praktikum

1. Memahami konsep Abstract Data Type (ADT) Stack dan Queue beserta karakteristik urutannya masing-masing.
2. Mengimplementasikan struktur data Stack dan Queue menggunakan Linked List dalam bahasa pemrograman Python.
3. Menganalisis kompleksitas waktu operasi inti Stack dan Queue, meliputi Push, Pop, Enqueue, dan Dequeue.
4. Membedakan karakteristik LIFO dan FIFO serta kasus penggunaan nyata dari masing-masing struktur data.

B. Dasar Teori

1. Abstract Data Type ADT

Abstract Data Type (ADT) adalah model matematis untuk suatu tipe data yang didefinisikan melalui perilakunya dari sudut pandang pengguna, bukan melalui detail implementasinya [1]. ADT memisahkan antara **antarmuka** (operasi apa yang tersedia dan semantiknya) dengan **implementasi** (bagaimana operasi itu dijalankan di memori), sehingga memungkinkan penggantian implementasi tanpa memengaruhi kode yang bergantung padanya [2].



Gambar 1: Queue vs Stack

2. Stack

Stack adalah ADT yang mengikuti disiplin akses *Last In, First Out* (LIFO): elemen yang paling terakhir dimasukkan adalah yang pertama kali dikeluarkan [1]. Operasi-operasinya meliputi Push, Pop, Peek, dan isEmpty.

3. Queue

Queue adalah ADT yang mengikuti disiplin akses *First In, First Out* (FIFO): elemen yang pertama kali dimasukkan adalah yang pertama kali dikeluarkan [1]. Operasi-operasinya meliputi Enqueue, Dequeue, dan isEmpty.

4. Linked List sebagai Basis Implementasi

Linked List adalah struktur data linear yang terdiri atas simpul-simpul (*node*) yang masing-masing menyimpan data dan sebuah pointer ke simpul berikutnya [1]. Karakteristik ini menjadikannya

pilihan alami sebagai basis implementasi Stack dan Queue karena alokasi memori dinamis dan operasi $O(1)$ pada ujung list.

C. Hasil dan Pembahasan

1. Implementasi Node

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
```

2. Implementasi Stack

```
src/stack.py
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Stack:
7     def __init__(self):
8         self.top = None
9         self.size = 0
10
11     def push(self, data):
12         node = Node(data)
13         node.next = self.top
14         self.top = node
15         self.size += 1
16
17     def pop(self):
18         if self.is_empty():
19             raise IndexError("Stack is empty")
20         data = self.top.data
21         self.top = self.top.next
22         self.size -= 1
23         return data
24
25     def peek(self):
26         if self.is_empty():
27             raise IndexError("Stack is empty")
28         return self.top.data
29
30     def is_empty(self):
31         return self.top is None
32
```

Berikut demonstrasi eksekusi Stack:

```
Eksekusi
s = Stack()
s.push(10); s.push(20); s.push(30)
print(s.pop())
```

```
Output
30
```

3. Implementasi Queue

```
src/queue.py
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6
7 class Queue:
```

```

8     def __init__(self):
9         self.front = None
10        self.rear = None
11        self.size = 0
12
13    def enqueue(self, data):
14        node = Node(data)
15        if self.rear is not None:
16            self.rear.next = node
17            self.rear = node
18        if self.front is None:
19            self.front = node
20        self.size += 1
21
22    def dequeue(self):
23        if self.is_empty():
24            raise IndexError("Queue is empty")
25        data = self.front.data
26        self.front = self.front.next
27        if self.front is None:
28            self.rear = None
29        self.size -= 1
30        return data
31
32    def is_empty(self):
33        return self.front is None
34

```

4. Analisis Kompleksitas

Tabel 1: Kompleksitas Waktu Stack dan Queue Berbasis Linked List

Operasi	Stack	Queue
Push / Enqueue	$O(1)$	$O(1)$
Pop / Dequeue	$O(1)$	$O(1)$
Ruang (Space)	$O(n)$	$O(n)$

D. Kesimpulan

1. Stack sebagai ADT LIFO berhasil diimplementasikan menggunakan Linked List.
2. Queue sebagai ADT FIFO memerlukan dua pointer `front` dan `rear`.
3. Linked List memberikan garansi $O(1)$ untuk operasi di ujung list.

Lampiran

FULL SOURCE CODE ada di Lampiran.

```
stack.py
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Stack:
7     def __init__(self):
8         self.top = None
9         self.size = 0
10
11     def push(self, data):
12         node = Node(data)
13         node.next = self.top
14         self.top = node
15         self.size += 1
16
17     def pop(self):
18         if self.is_empty():
19             raise IndexError("Stack is empty")
20         data = self.top.data
21         self.top = self.top.next
22         self.size -= 1
23         return data
24
25     def peek(self):
26         if self.is_empty():
27             raise IndexError("Stack is empty")
28         return self.top.data
29
30     def is_empty(self):
31         return self.top is None
32
```

Daftar Pustaka

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 4th ed. The MIT Press, 2022.
- [2] A. V. Aho, J. E. Hopcroft, dan J. D. Ullman, *Data Structures and Algorithms*. Addison-Wesley, 1983.
- [3] R. Sedgewick dan K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011.
- [4] A. Einstein, “Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies],” *Annalen der Physik*, vol. 322, no. 10, hlm. 891–921, 1905, doi: <http://dx.doi.org/10.1002/andp.19053221004>.
- [5] D. E. Knuth, “Fundamental Algorithms,” Addison-Wesley, 1973, chap. 1.2.